

Helix Thready → Figma import bundle

Contents

How the capture was produced — genuine, reproducible

What was attempted with the live daemon first (recorded evidence)

Operator steps — land it in Figma

VERIFIED vs ASSUMED

This folder is the **Figma import set** for the Helix Thready MVP design system. It contains **genuine** OD Figma capture IR — not a mock — produced headless from the live design pages, plus the brand SVGs and the exact operator steps to land them in Figma.

Outcome (VERIFIED): genuine `.od-figma.json` IR was produced headless and validated. The final step (opening the JSON *inside Figma desktop* to rebuild editable layers) is **operator-interactive** and **offline** — the OD Figma Import plugin declares

`networkAccess: { allowedDomains: ["none"] }`, so nothing here pushes to Figma's cloud, and **no Figma API token exists** in this environment. No cloud push is claimed or possible.

Contents

```
figma/
  thready.od-figma.json      combined super-root board — every page as an offset child frame
  captures/<area>-<screen>.od-figma.json  one genuine IR per page (28 files)
  capture-manifest.json      provenance + per-page validation (node counts, fonts, viewport)
  assets/*.svg               brand + component vectors (logo, launcher icons, slogan,
  components-sheet)
  IMPORT.md                  this file
```

Every `*.od-figma.json` conforms to **OD Figma capture IR v1** (`open-design/figma-plugin/IR.md`):

`{ version:1, source:{url,title,capturedAt,viewport,dpr}, fonts:[...], root:{ FRAME ... } }`. `root` is a node-tree of `FRAME` / `TEXT` / `RECTANGLE` nodes with absolute geometry, fills, strokes, corner radii, shadows and the fonts each TEXT run references.

How the capture was produced — genuine, reproducible

A native binary `.fig` **cannot** be produced outside Figma; the OpenDesign pipeline instead uses a JSON node-tree that a Figma plugin rebuilds via the Plugin API. The producer is OpenDesign's own clipper

runtime `open-design/clipper/capture.js`, which exposes `window.__odCapture()` returning `{ html, figmaIr, ... }`.

`scripts/capture-figma.mjs` drives Chromium headless over the DevTools Protocol and, per page: emulates light theme → navigates → **injects the clipper's own** `capture.js` → reads back `window.__odCapture({includeImages:true}).figmaIr`. Because every Thready page is self-contained (inline CSS/SVG, data-URI images, no cross-origin resources), the IR is **complete as captured** — the clipper's cross-origin worker-inlining step is a no-op here. This is the *same producer* the OD Clipper's *Download Figma* (*.json*) action and the daemon's `/api/library/assets/:id/figma` sidecar use — it is **not** hand-authored plugin output.

Reproduce:

```
bash scripts/capture-figma.sh      # → captures/*.od-figma.json, thready.od-figma.json, capture-
    manifest.json
```

What was attempted with the live daemon first (recorded evidence)

The daemon's advertised export `od library figma <assetId>` only serves IR for assets that already carry a **clipper capture sidecar** (`<contentHash>.od-figma.json`). Hand-authored HTML imported through the daemon never gets one. Verbatim, against daemon **v0.14.1** at `http://127.0.0.1:7456`:

Command	Result
<code>od library list --json</code>	1 pre-existing <code>html</code> asset, <code>sourceKind: "manual-upload"</code>
<code>od library figma <existing-id> --out ...</code>	<code>{"error":{"code":"NOT_FOUND","message":"no figma capture for this asset"}}</code>
<code>od library import library/components.html --json</code>	created asset, <code>sourceKind: "manual-upload"</code>
<code>od library figma <new-id> --out ...</code>	<code>{"error":{"code":"NOT_FOUND","message":"no figma capture for this asset"}}</code>
<code>od tools design-systems read --path ...</code>	<code>{"ok":false,"error":{"message":"OD_TOOL_TOKEN is required"}}</code> (agent-run only)

Conclusion: the daemon **cannot mint the IR from imported HTML** — the IR is a *clipper* artifact. Rather than stop, the capture was produced by running the clipper's real `capture.js` headless (above): faithful to the intended pipeline, fully genuine.

Operator steps — land it in Figma

`.od-figma.json` is **not** a Figma file — dragging it into Figma shows *“Unsupported file format”* (expected). It is imported through the **OD Figma Import** plugin.

1. Install the plugin once (Figma desktop app required): 1. Open the Figma **desktop** app; open or create any file. 2. Menu → **Plugins → Development → Import plugin from manifest...** 3. Select `open-design/figma-plugin/manifest.json`. The plugin now lives under **Plugins → Development → OD Figma Import**.

2. Import a capture (repeat per page): 1. Run **Plugins → Development → OD Figma Import**. 2. In the plugin window, **drop or choose** a `.od-figma.json` — `thready.od-figma.json` for the whole board, or any `captures/*.od-figma.json` for one screen — or paste the JSON. 3. Click **Import**. It rebuilds as a named frame, selected and zoomed to fit.

The `assets/*.svg` vectors can be dragged straight onto the canvas (native Figma SVG import) as editable vectors — no plugin needed. This matters because **in-page `<svg>` is excluded from the IR** (`capture.js` `SKIP_TAGS` includes `SVG`), so brand marks rendered as inline SVG are not in the node tree — import them from `assets/` separately.

VERIFIED vs ASSUMED

- **VERIFIED** — 28 per-page IRs + the combined `thready.od-figma.json` exist, parse as JSON, and match IR v1 top-level shape (`version/source/fonts/root:FRAME`). Per-page + combined node counts and validation are in `capture-manifest.json` (combined board: 28 pages, 11,292 nodes).
- **VERIFIED** — the daemon cannot emit IR for manual-upload/imported HTML (evidence table above).
- **ASSUMED (not executed here)** — the in-Figma rebuild fidelity: it needs the Figma desktop app + an interactive plugin run, not exercisable headless. The plugin (`figma-plugin/code.js`) consumes exactly this IR shape. Per its README: live geometry preserved; fonts load when available else fall back to *Inter Regular*; SVGs rasterized to PNG on import; complex CSS simplified.
- **networkAccess = "none", no Figma token** — no cloud push occurs or is possible. `[OPEN]` a cloud round-trip would require an operator-interactive Figma desktop session.

Made with love ♥ by Helix Development.